

Class 14 — Topic 4: Collecting and Analyzing Monitoring Data

- 4.1 Event logs: Windows and Linux
- 4.2 Network traffic capture: the basics
- 420-1C1-DW Preventative Monitoring — LEA.D8 AEC — Term 1
- Friday, October 9, 2026 — online, 6:00–10:00 pm
- Instructor: Asen Grozdanov — Cyber.SoHo Educational Hub
- Competency EZ01, element 4 — performance criterion 4.2: methodical analysis of event logs and network traffic



Before we read or run anything — the legal frame

- Educational material — Cyber.SoHo / Asen Grozdanov — © 2026, all rights reserved
- Use only in an isolated, authorized lab, against Designated Targets
- Without the owner's written authorization, the same techniques are offences: Criminal Code ss. 342.1, 342.2, 430(1.1), 184
- Section 7.2 of the Legal Notice: monitoring, capture and surveillance data
 - capture only on systems you own or on Designated Targets
 - every log and capture is personal information: minimum, protected, deleted after marking
- Full text: Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy v2.0
- <https://cyber-soho.github.io/prevmon/00.01.%20cyber.soho%20-%20legal.html>



Where we are in the course

- Topics 1–3: foundations, the network to monitor, the tool, the baseline (3.4), the alerts (3.5)
- Lab 1: map and monitoring plan — Lab 2: tool installed, baseline built, alerts set
- Topic 4: the data the tool collects — today 4.1 and 4.2
 - 4.1 event logs (host-based evidence)
 - 4.2 network traffic capture (network-based evidence)
- Next: 4.3 network files and system activity, 4.4 filtering and searching, 4.5 normal vs abnormal
- Saturday Oct 10: Topic 5.1–5.2 and Quiz 3 (5 %) — Friday Oct 16: Lab 3 (5 %)



Learning outcomes of this class

- 1. Explain what an event record contains; name the Windows log channels and the Linux log files
- 2. Recognize the Windows Security event IDs and the Linux authentication messages a technician reads first
 - (failed logon vs unknown account vs locked account)
- 3. Explain what a packet capture is, where it can be taken, and capture filter vs display filter
- 4. Produce two first-level summaries from raw data: failed logons per source; conversations per host
- Each lesson: concepts → videos → real-life example → script with line-by-line commentary → flash cards → quiz



4.1 Event logs: Windows and Linux

- What an event log is and why monitoring depends on it
- Windows: channels, event anatomy, audit policy, event IDs, logon types, tools, retention
- Linux: syslog and journald, facilities and severities, files, tools, rotation, forwarding
- Rules for both platforms
- Example 4.1: Monday-morning log review at Atelier Nordik inc.

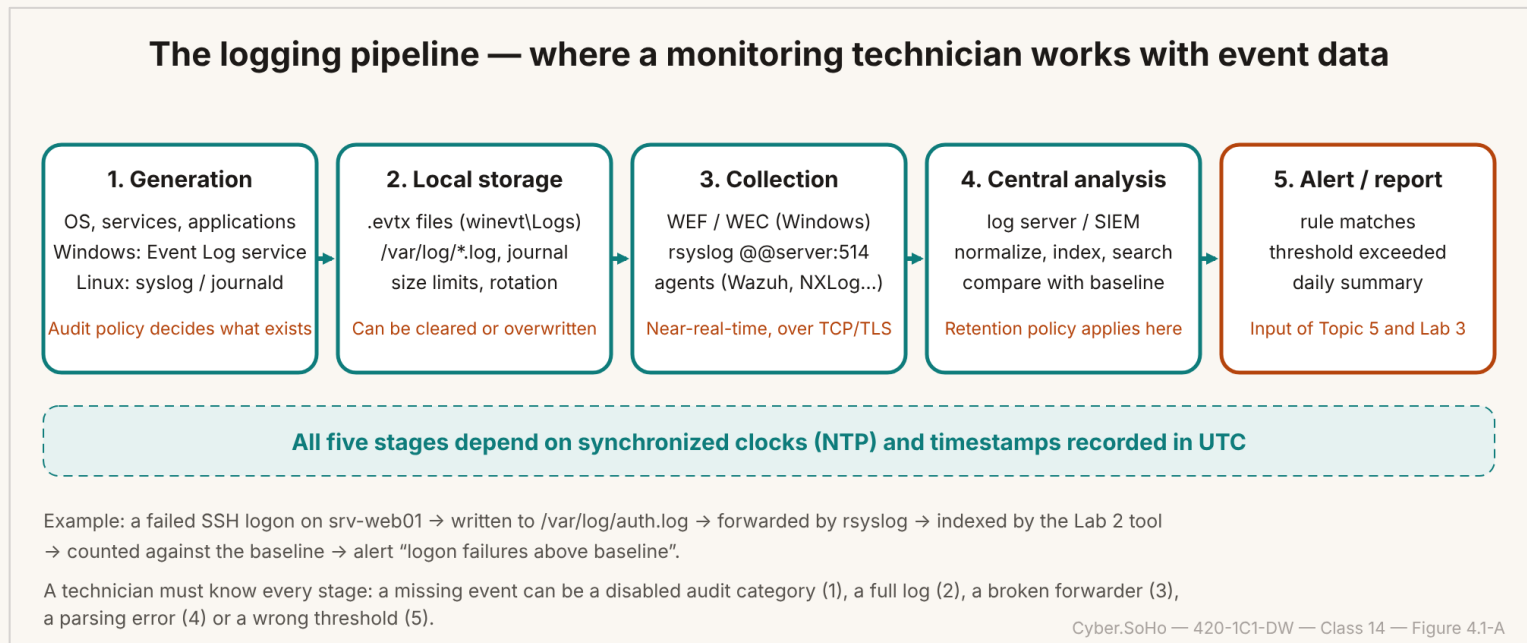


What an event log is

- Event log = time-ordered file where the OS, services and applications record events
- Event record (log entry) — at least six fields:
 - timestamp (with time zone)
 - source (which component wrote it)
 - identifier: Windows Event ID / Linux program name + PID
 - severity level
 - host
 - message: user, address, result
- Operational logs (health: errors, services) vs audit logs (who did what: logons, privileges, changes)
- Completeness is decided before an incident: what was never written cannot be found

The logging pipeline — Figure 4.1-A

- 1 Generation (OS, services, applications; audit policy decides what exists)
- 2 Local storage (.evtx, /var/log, journal; size limits, rotation)
- 3 Collection (WEF/WEC, rsyslog @@server:514, agents)
- 4 Central analysis (log server / SIEM; baseline comparison; retention)
- 5 Alert / report (input of Topic 5 and Lab 3)
- All five stages depend on synchronized clocks (NTP) and timestamps in UTC



Windows Event Log: the channels

- Windows Event Log service → binary .evtx files in C:\Windows\System32\winevt\Logs\
- Read with Event Viewer (eventvwr.msc), wevtutil, PowerShell Get-WinEvent
- Windows Logs: Application · Security (audit, written only by LSASS) · System · Setup · Forwarded Events
- Applications and Services Logs: hundreds of channels, each with Admin / Operational / Analytic / Debug
 - PowerShell/Operational, Windows Defender/Operational, TaskScheduler/Operational
 - TerminalServices-LocalSessionManager/Operational (remote desktop sessions)
 - Sysmon/Operational when System Monitor is installed (process + command line + hash, network connections)



Anatomy of a Windows event — Figure 4.1-B

- Event Viewer fields: Level (or Keywords Audit Success / Audit Failure), Date and Time, Source, Event ID, Task Category, User, Computer, Description
- Details tab = XML: <System> (identifiers, time in UTC) and <EventData> (named fields)
- EventData of 4625: TargetUserName, IpAddress, IpPort, LogonType, Status, SubStatus
- The same question on Linux: one auth.log line — timestamp, host, program[PID], message
- Both answer: who, from where, when, result

Same question, two formats: “who failed to log on, from where, when?”

Windows — Security log, Event ID 4625 (XML view)

```
<Event>
  <System>
    <Provider Name="Microsoft-Windows-Security-Auditing"/>
    <EventID>4625</EventID>
    <Keywords>Audit Failure</Keywords>
    <TimeCreated SystemTime="2026-10-04T06:40:12Z"/>
    <Computer>SRV-AD01.nordik.lan</Computer>
  </System>
  <EventData>
    <Data Name="TargetUserName">administrator</Data>
    <Data Name="Status">0xC000006D</Data>
    <Data Name="SubStatus">0xC000006A</Data>
    <Data Name="LogonType">3</Data>
    <Data Name="IpAddress">10.8.0.6</Data>
    <Data Name="IpPort">52110</Data>
  </EventData>
</Event>
```

SystemTime is always UTC ("Z"); Event Viewer shows local time.
SubStatus 0xC000006A = wrong password; 0xC0000064 = unknown user name.
LogonType 3 = network logon (SMB, remote management); 10 = RDP session.

Linux — /var/log/auth.log (rsyslog, RFC 3164 line)

```
Oct 4 03:14:07 srv-web01 sshd[2317]:
Failed password for invalid user admin
from 203.0.113.57 port 41022 ssh2
```

Field by field

- ① timestamp — month, day, local time, no year, no time zone
- ② hostname — which machine wrote the line
- ③ program[PID] — sshd, PID 2317 (tag)
- ④ message — free text written by the program
“invalid user” → the account does not exist
“Failed password for root” → account exists, wrong password

The same line in RFC 5424 format (journald / modern rsyslog):

```
<38>1 2026-10-04T03:14:07.512-04:00 srv-web01 sshd 2317 - -
Failed password for invalid user admin from 203.0.113.57 ...
```

PRI 38 = facility 4 (auth) × 8 + severity 6 (informational).
RFC 5424 adds the year, milliseconds and the UTC offset (-04:00 in Montréal).
Both formats carry the same four answers: who, from where, when, result.

Cyber.SoHo — 420-1C1-DW — Class 14 — Figure 4.1-B



Nothing is logged unless the audit policy says so

- The Security log contains only the categories enabled by the audit policy
- Configure: Local Security Policy or a GPO → Advanced Audit Policy Configuration
- Verify: `auditpol /get /category:*`
- Three settings for a small network:
 - Audit Logon — success and failure → 4624 / 4625
 - Audit Account Management → 4720–4756
 - Audit Process Creation → 4688 (+ "Include command line in process creation events")
- More categories = more volume → the next question is always the size of the log



Security event IDs that matter first (1/2) — logon family

Event ID	Log channel	What it records	Why it matters for monitoring
4624	Security	An account was successfully logged on	Baseline of who logs on, when and from where; unusual hours or hosts
4625	Security	An account failed to log on	Password guessing, locked accounts, misconfigured services; the input of Example 4.1
4634 / 4647	Security	An account was logged off / user-initiated logoff	Session duration; pairs with 4624
4648	Security	A logon was attempted using explicit credentials	"Run as" another account; lateral movement indicator
4672	Security	Special privileges assigned to new logon	Administrative logon; must match known admin activity
4688	Security	A new process has been created	Which programs run (needs "Audit Process Creation"; command line needs an extra policy)
4698 / 4702	Security	Scheduled task created / updated	Persistence: tasks created outside change windows
4720 / 4726	Security	User account created / deleted	Unexpected accounts on servers
4722 / 4725	Security	User account enabled / disabled	Dormant accounts re-enabled
4728 / 4732 / 4756	Security	Member added to a global / local / universal security group	New administrators; privilege changes
4740	Security	A user account was locked out	Many lockouts = guessing or a stale password in a service
4768 / 4769 / 4771	Security	Kerberos TGT requested / service ticket requested / pre-authentication failed	Domain logon activity; 4771 bursts = guessing against domain accounts



Security event IDs that matter first (2/2) — changes and tampering

Event ID	Log channel	What it records	Why it matters for monitoring
4776	Security	The computer attempted to validate the credentials (NTLM)	NTLM use where Kerberos is expected; failures = guessing
4719	Security	System audit policy was changed	Someone reduced what is logged — alert
1102	Security	The audit log was cleared	Evidence removal — alert immediately
1100 / 1104	Security	Event logging service shut down / log is full	Gaps in evidence
7045	System	A new service was installed	New services on servers are rare; persistence indicator
7034 / 7036	System	Service crashed / service state changed	Availability problems (performance vs. security, Topic 1.5)
6005 / 6006	System	Event Log service started / stopped	Boot and shutdown markers
41	System (Kernel-Power)	System rebooted without a clean shutdown	Crashes, power loss
1074	System (User32)	A process initiated a shutdown or restart	Who rebooted the server, and why
104	System	A log file was cleared	Evidence removal in System/Application logs



Logon types in 4624 and 4625 — Table 4.1-B

Logon type	Name	Meaning	Example
2	Interactive	Logon at the keyboard and screen of the computer	User signs in at the console
3	Network	Logon from the network, no interactive session	Opening a shared folder (SMB), remote management, many web services
4	Batch	Logon by a scheduled task	Task Scheduler job running as a service account
5	Service	Logon by a Windows service at start-up	SQL Server service starting
7	Unlock	Workstation unlocked	User returns from lunch and unlocks the screen
8	NetworkCleartext	Network logon with a password sent in clear text	Basic authentication on IIS — should be rare
9	NewCredentials	Same user, new credentials for outbound connections	runas /netonly
10	RemoteInteractive	Remote Desktop (RDP) or Terminal Services session	Administrator connects with Remote Desktop
11	CachedInteractive	Logon with cached domain credentials	Laptop signs in while away from the office



Why the logon failed — Sub Status codes of 4625 — Table 4.1-C

Sub Status	Meaning	Interpretation for monitoring
0xC000006A	Wrong password for an existing account	A few = typing errors; many = password guessing
0xC0000064	User name does not exist	Several in a row = someone is guessing account names
0xC0000234	Account locked out	Lockout policy triggered — see also 4740
0xC0000072	Account disabled	Use of an account that should be dead
0xC000006F	Logon outside the allowed hours	Policy violation or automated activity at night
0xC0000070	Logon from an unauthorized workstation	Workstation restriction is working
0xC0000071	Password expired	Operational: user must change password
0xC0000133	Clock difference too large between client and server	Time synchronization problem (Kerberos needs < 5 min skew)



Reading the log: Event Viewer, wevtutil, PowerShell

- Event Viewer → Filter Current Log: event IDs (4625,4740) and a time range
- `wevtutil qe Security /q:"*[System[(EventID=4625)]]" /c:20 /rd:true /f:text`
- `Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4625; StartTime=(Get-Date).AddHours(-24)}`
- Get-EventLog is the legacy cmdlet (Windows PowerShell 5 only, slow on large logs)
- | `Export-Csv -NoTypeInfo security_export.csv` → the input of `logon_failures.py`
- Companion script tonight: `export_security_4625.ps1` (extracts TargetUserName, IpAddress, LogonType, SubStatus)



Size, retention, forwarding

- Default: each standard log $\approx$ 20 MB, then "overwrite events as needed" — a DC can wrap in minutes
- Measure 1: increase the maximum size (hundreds of MB on servers)
- Measure 2: forward continuously to a central store
 - Windows Event Forwarding (WEF) + Windows Event Collector (WEC): built-in, agentless, WinRM
 - Agents: Wazuh agent, Winlogbeat, NXLog, Splunk forwarder
- Forwarding protects evidence: 1102 cannot erase the copy that already left — and 1102 itself raises an alert
- The Lab 2 tool is the central store of this slide

Linux: two logging systems that run together

- syslog family — rsyslog (sometimes syslog-ng): plain-text lines in /var/log/
- systemd-journald: binary, indexed journal read with journalctl
- Ubuntu/Debian: journald collects → forwards to rsyslog → rsyslog writes the files
- RHEL family: rsyslog installed, journal is the primary store
- Same vocabulary for both: facility, severity, PRI, timestamp, host, program[PID], message

The syslog message: facility, severity, PRI — RFC 5424 / RFC 3164

- Facility = which subsystem: kern 0, user 1, mail 2, daemon 3, auth 4, syslog 5, cron 9, authpriv 10, local0–7 = 16–23
- Severity = how serious: 0 emerg, 1 alert, 2 crit, 3 err, 4 warning, 5 notice, 6 info, 7 debug
- PRI = facility $\times$ 8 + severity $\rightarrow$ auth (4) + info (6) = 38
- rsyslog rule: auth,authpriv.* /var/log/auth.log
- Classic line: Oct 4 03:14:07 srv-web01 sshd[2317]: Failed password for invalid user admin from 203.0.113.57 port 41022 ssh2
 - timestamp (no year, no zone) · hostname · program[PID] · free-text message
- Program vocabulary: sshd "Accepted"/"Failed password"/"Invalid user", sudo "COMMAND=", pam_unix "session opened for user root"

Where the files are — Table 4.1-D

Content	Debian / Ubuntu	RHEL / Rocky / AlmaLinux	Read with
General system messages	/var/log/syslog	/var/log/messages	less, tail -f, grep
Authentication: ssh, sudo, su, PAM	/var/log/auth.log	/var/log/secure	grep "Failed password", grep sudo
Kernel messages	/var/log/kern.log and dmesg	dmesg, journalctl -k	dmesg -T
Boot messages	/var/log/boot.log (if enabled)	/var/log/boot.log	journalctl -b
Scheduled jobs (cron)	in syslog (or /var/log/cron.log)	/var/log/cron	grep CRON
Package installations	/var/log/dpkg.log, /var/log/apt/history.log	/var/log/dnf.log	less
Web server access and errors	/var/log/apache2/ or /var/log/nginx/	/var/log/httpd/ or /var/log/nginx/	tail -f access.log
Linux Audit framework (auditd)	/var/log/audit/audit.log	/var/log/audit/audit.log	ausearch, aureport
Successful logins (binary)	/var/log/wtmp	/var/log/wtmp	last
Failed logins (binary)	/var/log/btmp	/var/log/btmp	lastb (root only)
Last login per user (binary)	/var/log/lastlog	/var/log/lastlog	lastlog
Currently logged-in users	/run/utmp	/run/utmp	who, w
systemd journal (binary, all units)	/var/log/journal/ or /run/log/journal/	/var/log/journal/	journalctl

Reading the logs: everyday commands

- `tail -f /var/log/auth.log` — follow live
- `grep "Failed password" /var/log/auth.log | wc -l` — count failures
- `grep -c sudo /var/log/auth.log` — count sudo lines
- `journalctl -u ssh --since "1 hour ago"` — one service, one hour
- `journalctl -p err -b` — errors since boot · `journalctl -k` — kernel
- `last / lastb` — successful and failed logins
- `logger -p auth.warning "monitoring test"` — write a test event and look for it in the central tool

Rotation and forwarding

- logrotate: /etc/logrotate.conf, /etc/logrotate.d/ → auth.log, auth.log.1, auth.log.2.gz ...
retention count
- journald: SystemMaxUse= in /etc/systemd/journald.conf; journalctl --disk-usage
- rsyslog forwarding: *.* @@logsrv.nordik.lan:514
 - @ = UDP (fast, no guarantee) · @@ = TCP (reliable, base for TLS)
- Forwarding is also integrity: a copy that has left the host cannot be erased from the host

Linux messages to watch — Table 4.1-G

Message (pattern)	File	Meaning	Why it matters
sshd: Accepted password/publickey for USER from IP	auth.log / secure	Successful SSH logon	Baseline of who connects; unknown IP or user = anomaly
sshd: Failed password for [invalid user] USER from IP	auth.log / secure	Failed SSH logon	Repeated = password guessing (Example 4.1)
sshd: Invalid user USER from IP	auth.log / secure	Account name does not exist	Name guessing
sudo: USER : TTY=... ; COMMAND=/...	auth.log / secure	A command run with sudo	Who ran what as root
pam_unix(...): authentication failure; ... rhost=IP user=USER	auth.log / secure	PAM refused a logon	Generic failure counter
session opened for user root	auth.log / secure	Root session started	Root use must match change records
useradd / groupadd: new user / new group	auth.log / secure	Account or group created	Unexpected accounts
systemd: Failed to start UNIT	syslog / messages	A service did not start	Availability (performance issue)
kernel: Out of memory: Killed process	kern.log / messages	Memory exhaustion	Availability; sometimes a symptom of abuse
kernel: eth0: Link is Down / Up	kern.log / messages	Network link change	Cable, switch or interface problem
CRON: (root) CMD (...)	syslog / cron	Scheduled job executed	Unknown jobs = persistence
auditd: type=SYSCALL ... key="..."	audit/audit.log	A watched system call or file access	File integrity and privileged actions

Rules that apply to both platforms

- Time: NTP everywhere (w32tm, chrony/timesyncd); compare in UTC; Montréal changes zone twice a year
- Completeness: enable logon, account management, process creation, sudo before you need them; verify with a test event
- Centralization and integrity: forward promptly, restrict reading, keep the local copy, alert on 1102/4719/1100 and on a stopped rsyslog
- Retention: e.g. 90 days online, 1 year archived — NIST SP 800-92, Guide to Computer Security Log Management
- Privacy: a log line is personal information (Law 25; Legal Notice 6.7 and 7.2) — minimum, protected, deleted
- Volume: a workstation = thousands of events/day, a DC = millions — collect what you can analyze; the baseline tells you what is normal

Videos for 4.1 — verified, each under 15 minutes

- Professor Messer — Log Data (Security+ SY0-701, 4.9) — 13:41 — youtube.com/watch?v=EDru1LTyDJw
- ITOpsTalk (Microsoft community) — Understanding the Windows Server Event Log — ≈ 13:40 — youtube.com/watch?v=2uVX5yalq2E
- Server Academy — Event Viewer & Windows Logs — ≈ 7:20 — youtube.com/watch?v=X0WDzktpr5I
- Red Hat Enterprise Linux — Understanding Linux Logging: /var/log, Journal, Logger — ≈ 6:35 — youtube.com/watch?v=SFdcbbdhOs90
- Watch at home; check every link with VirusTotal before clicking (Legal Notice, Annex C)

Example 4.1 — Monday-morning log review at Atelier Nordik inc.

- Fictional 14-person design firm, Mile-End: OPNsense firewall, LAN 192.168.10.0/24, DMZ 192.168.20.0/24, VPN pool 10.8.0.0/24
- SRV-AD01 (192.168.10.5, AD DS + files) · srv-web01 (192.168.20.10, Ubuntu) · monitoring server 192.168.10.250
- Monday 08:05: alert "logon failures above baseline" for the weekend
- Task: count failed logons per (host, source) in the Windows Security log and the Linux auth.log; one table for the team lead
- Inputs: security_export.csv (export_security_4625.ps1) + auth.log
- Rule: 10 or more failures inside any 10-minute window → ALERT
- Output: logon_failures_summary.csv — the detection evidence of Lab 3

Example 4.1 — what logon_failures.py does, step by step

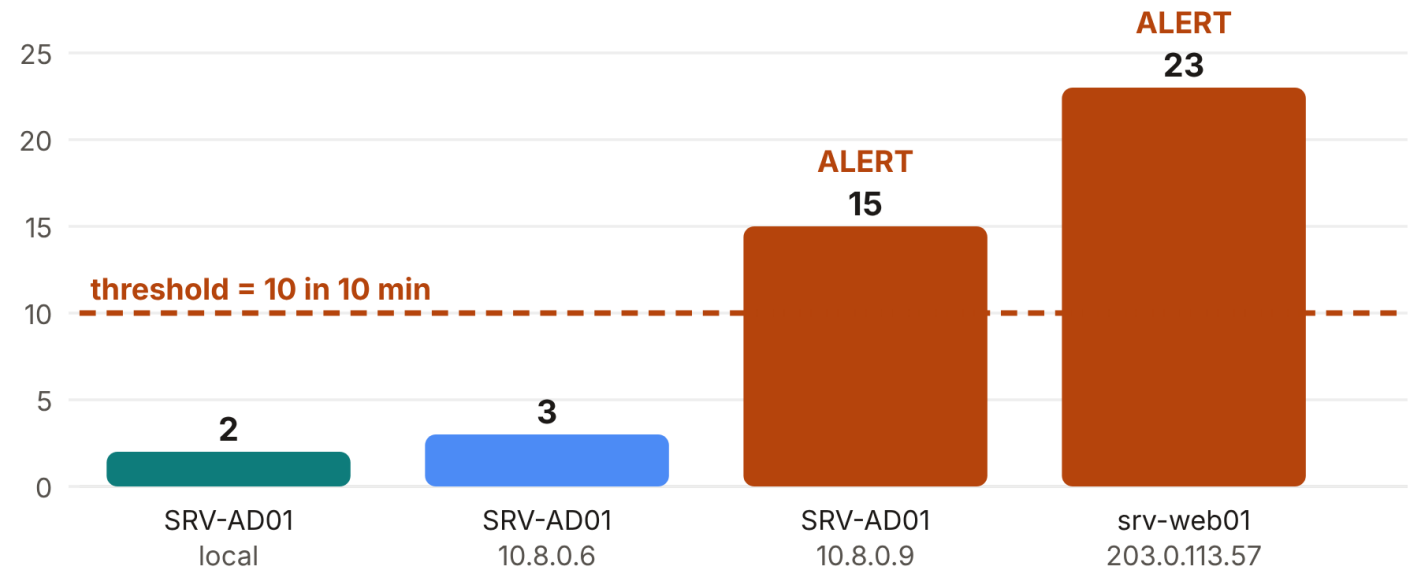
- 1 read_windows(): CSV rows → keep Id 4625 → parse ISO time → "-" becomes "local" (console/service logon)
- 2 read_linux(): regular expression on "Failed password for [invalid user] NAME from A.B.C.D" → add the year
- 3 check_window(): sort times, slide a 10-minute window, keep the largest count
- 4 summarize(): group by (host, source) → attempts, first/last, user names, max in 10 min, flag
- 5 main(): write logon_failures_summary.csv, print the aligned table, count the flagged sources
- Python 3, standard library only; a comment before every line of code in the module



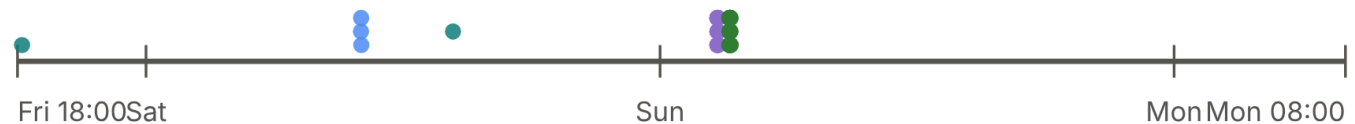
Example 4.1 — the result, and what it does not say

- srv-web01 ← 203.0.113.57 (Internet): 23 attempts, 7 account names, 03:14–03:18 Sunday → ALERT
- SRV-AD01 ← 10.8.0.9 (VPN pool): 15 attempts on "administrator", 02:40–02:42 Sunday → ALERT
- SRV-AD01 ← 10.8.0.6: 3 attempts (ldube, Saturday 10:02) → normal
- SRV-AD01 ← local: 2 attempts (jtremlay console, backup-svc task) → normal
- The summary says WHERE to look, not WHY: which VPN account held 10.8.0.9? did any logon succeed? was the account locked?
- Those questions — and the raw files as evidence — are Lab 3

Failed logons per (host, source) — threshold: 10 within 10 minutes



Timeline of the attempts (Fri 18:00 → Mon 08:00)



Check-in 4.1 — answer aloud

- Which log channel records logons, and which two event IDs?
- Ubuntu vs RHEL: where are SSH and sudo messages?
- What is Logon Type 10?
- Compute the PRI of an auth message of severity informational
- Why NTP, and in which time zone do we compare logs?
- In the module: 5 flash cards, 4 multiple-choice and 2 multiple-select questions with hints



4.2 Network traffic capture: the basics

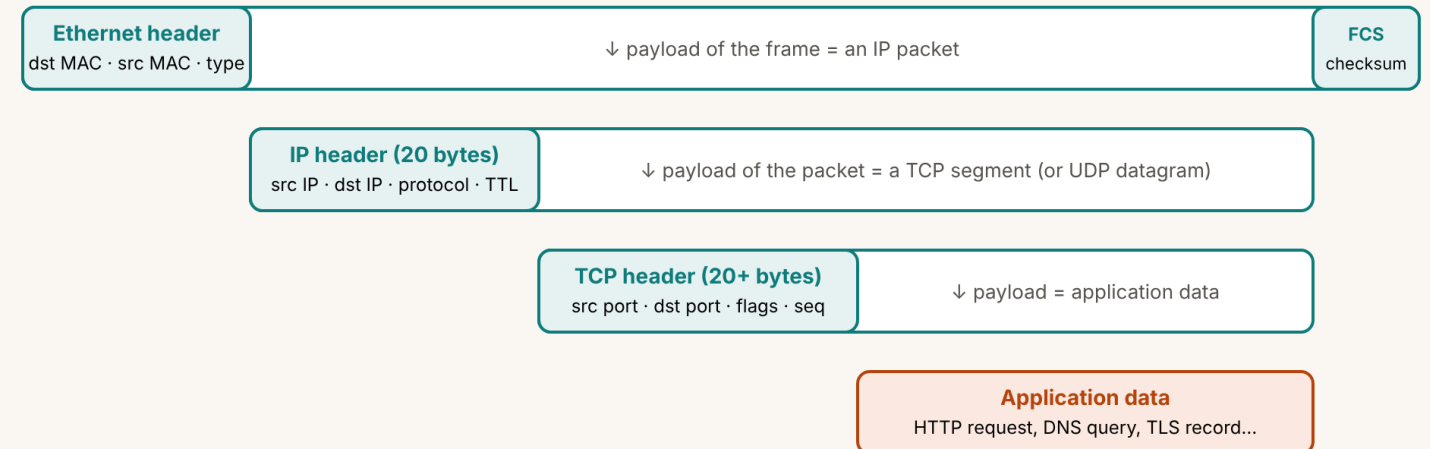
- Frames, packets, ports, flags, conversations
- What a packet capture is and where to take it
- Full capture vs flows vs metadata; tools
- Capture filters vs display filters; the commands
- Reading a capture for monitoring; legal limits
- Example 4.2: evening traffic from workstation WS-10-44



What travels on the wire — encapsulation — Figure 4.2-A

- Application data → TCP segment / UDP datagram (ports) → IP packet (addresses, protocol, TTL) → Ethernet frame (MAC addresses)
- Each layer adds a header; an analyzer decodes them layer by layer
- Ethernet: which device (MAC) — IP: which hosts, which protocol (6 TCP, 17 UDP, 1 ICMP)
- TCP/UDP: which service (port) and connection state (SYN, ACK, FIN, RST)
- Data: readable only when not encrypted

One frame on the wire = four nested units; each header answers a monitoring question



What each layer tells the monitor

Ethernet: which physical device (MAC address) sent the frame — an unknown device on the LAN shows up here (ARP, DHCP).

IP: which hosts talk (internal ↔ internal, internal ↔ Internet) and which protocol (6 = TCP, 17 = UDP, 1 = ICMP).

TCP/UDP: which service (port 22, 53, 80, 443, 445, 3389) and the connection state (SYN, SYN-ACK, ACK, RST).

Application data: readable only when not encrypted; with TLS you still see sizes, timing and the SNI server name.

Cyber.SoHo — 420-1C1-DW — Class 14 — Figure 4.2-A

The header fields a monitor reads — Table 4.2-A

Number	Protocol / service	Transport	What its presence usually means
proto 1	ICMP (Internet Control Message Protocol)	IP	ping, errors; many pings to many hosts = discovery
proto 6	TCP (Transmission Control Protocol)	IP	Connection-oriented traffic (web, SSH, SMB...)
proto 17	UDP (User Datagram Protocol)	IP	Connectionless traffic (DNS, DHCP, NTP, VoIP)
22	SSH (Secure Shell)	TCP	Remote administration of Linux and network devices
23	Telnet	TCP	Clear-text remote shell — should not exist in 2026
25 / 587	SMTP (Simple Mail Transfer Protocol)	TCP	Mail sending; workstations sending on 25 = suspicious
53	DNS (Domain Name System)	UDP/TCP	Name resolution; must go only to the internal resolver
67 / 68	DHCP (Dynamic Host Configuration Protocol)	UDP	Address assignment; rogue DHCP server = anomaly
80	HTTP (Hypertext Transfer Protocol)	TCP	Clear-text web
123	NTP (Network Time Protocol)	UDP	Time synchronization
135 / 139 / 445	RPC / NetBIOS / SMB (Server Message Block)	TCP	Windows file sharing and management; SMB to the Internet = block and alert
389 / 636	LDAP / LDAPS (Lightweight Directory Access Protocol)	TCP	Directory queries to the domain controller
443	HTTPS (HTTP Secure, TLS)	TCP	Encrypted web; most Internet traffic
514	Syslog	UDP/TCP	Log forwarding to the log server
3389	RDP (Remote Desktop Protocol)	TCP	Remote desktop; exposure to the Internet is a classic risk
4444 / 1337 / high ports	No registered service	TCP	Unknown destinations; needs diagnosis



The TCP three-way handshake — intent and result in three packets

- Client SYN → Server SYN-ACK → Client ACK → data → FIN or RST
- SYN answered by SYN-ACK: port open, session started
- SYN answered by RST: port closed
- SYN with no answer: nothing listens, or a firewall dropped it
- One source, many ports of one host, mostly RST = port scan (Example 4.2)
- One source, one port, many hosts = sweep



What encryption hides, and what it does not

- TLS protects the application data: the payload cannot be read by the monitor
- Still visible: addresses, ports, number and size of packets, timing
- Usually visible: the SNI (Server Name Indication) of the first TLS message — the server the client asked for
- Monitoring remains possible: who talks to whom, how much, how often, when — the values of the baseline



What a packet capture is

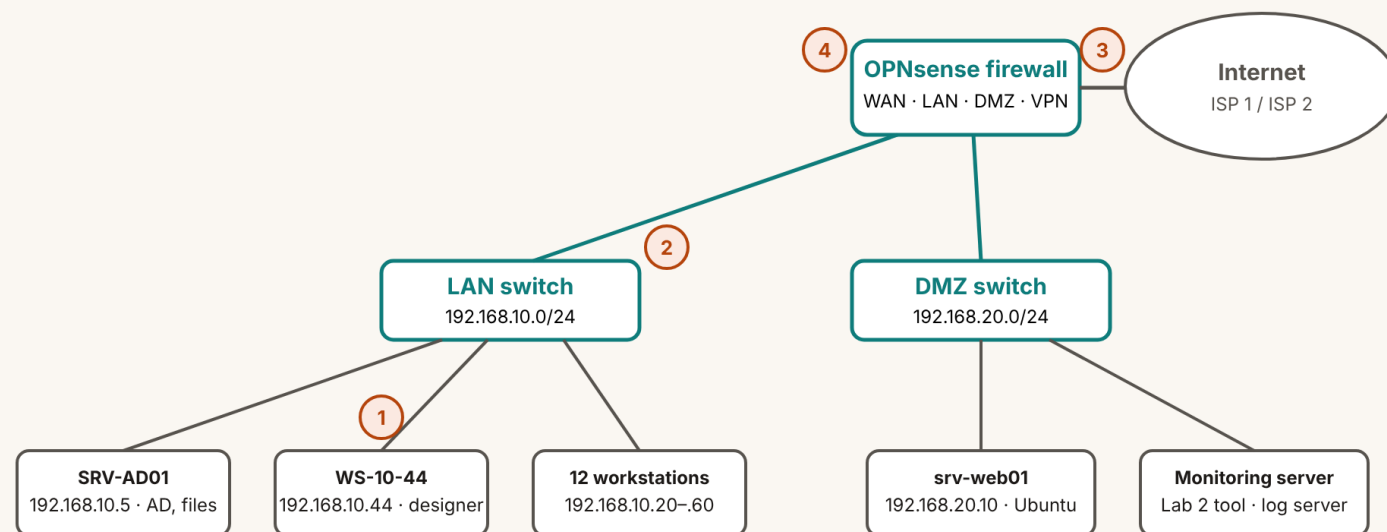
- Copy of every frame seen by an interface, with a timestamp, in memory or in a pcap / pcapng file
- Capture library: libpcap (Linux, macOS), Npcap (Windows); tools sit on top (Wireshark, tcpdump)
- Promiscuous mode: the NIC passes all frames, not only those addressed to its MAC
- But a switch forwards a unicast frame only to the port of its destination MAC
- → a workstation in promiscuous mode sees its own traffic and broadcasts, nothing else
- → to observe other hosts, bring their traffic to your interface



Where to capture — four points — Figure 4.2-B

- 1 On the host under study: quickest, complete for that host, but modifies the machine; impossible on appliances
- 2 SPAN / mirror port: the switch copies chosen ports or VLANs to the monitoring port — the usual choice; a saturated switch may drop frames
- 3 Hardware TAP: passive, inline (typically on the Internet link), exact copy, cannot be disabled by software
- 4 Firewall capture (OPNsense / pfSense): everything crossing the gateway, nothing that stays inside one switch
- Rule: capture as close as possible to the host you investigate; avoid tools on a busy production server

Where can a technician capture traffic? Four capture points on the Atelier Nordik network



- 1 On the host** — Wireshark, tcpdump or Pktmon on WS-10-44: only that host's traffic and broadcasts; quick, but it changes the machine under study.
- 2 SPAN / mirror port** — the switch copies the traffic of chosen ports to the monitoring server's port; the usual choice in a small office.
- 3 Network TAP** — passive hardware inserted on the Internet link: exact copy, no load on the switch, cannot be switched off remotely.
- 4 Firewall capture** — OPNsense / pfSense "Packet Capture": everything crossing LAN, DMZ and Internet; nothing that stays inside one switch.

Cyber.SoHo — 420-1C1-DW — Class 14 — Figure 4.2-B

Full capture, flow records, metadata

- Full capture of a 100 Mbit/s link fully used ≈ 12.5 MB/s ≈ 45 GB per hour
- Flow records (NetFlow, IPFIX, sFlow): one record per conversation — who, with whom, port, bytes, when; no payload
- Network metadata from a sensor: Zeek conn.log, dns.log, ssl.log; Suricata eve.json
- Small networks keep flows or metadata continuously and run full captures on demand
- Limit a full capture: a capture filter + a ring buffer: `dumpcap -b filesize:102400 -b files:20`



Tools — Table 4.2-C

Tool	Platform	Kind	Typical use in a small network
Wireshark	Windows, Linux, macOS	Graphical analyzer (+ capture)	Open pcap files, Statistics > Conversations, Follow TCP stream
tshark	Windows, Linux, macOS	Command-line Wireshark	Export fields to CSV, statistics in scripts (-z conv,tcp)
dumpcap	Windows, Linux, macOS	Capture engine of Wireshark	Long captures with ring buffers (-b filesize, -b files)
tcpdump	Linux, macOS, BSD	Command-line capture	Quick capture on a server or firewall with a BPF filter
Npcap	Windows	Capture driver	Required by Wireshark on Windows (replaces WinPcap)
Pktmon (Packet Monitor)	Windows 10/11, Server 2019+	Built-in capture	Capture without installing anything; convert .etl to pcapng
OPNsense / pfSense Packet Capture	Firewall	Built-in capture	Capture at the gateway from the web interface
Zeek	Linux sensor	Network metadata logs	conn.log, dns.log, ssl.log kept for weeks
Suricata	Linux sensor / firewall	IDS/IPS + metadata (eve.json)	Alerts on signatures, flow records
NetFlow / IPFIX / sFlow collectors	Routers, switches, firewalls	Flow records	Who talked to whom and how much, without payload

Capture filters versus display filters — Table 4.2-B

Goal	Capture filter (tcpdump, dumpcap, Wireshark capture options)	Display filter (Wireshark, tshark -Y)
Everything to or from one host	host 192.168.10.44	ip.addr == 192.168.10.44
Only traffic sent by one host	src host 192.168.10.44	ip.src == 192.168.10.44
One subnet	net 192.168.20.0/24	ip.addr == 192.168.20.0/24
One service port (both TCP and UDP)	port 53	tcp.port == 53 udp.port == 53
Only HTTPS	tcp port 443	tcp.port == 443 or tls
Exclude your own SSH session	not port 22	!(tcp.port == 22)
Connection attempts only (SYN without ACK)	'tcp[tcpflags] & (tcp-syn tcp-ack) == tcp-syn'	tcp.flags.syn == 1 && tcp.flags.ack == 0
Resets (closed ports, aborted sessions)	'tcp[tcpflags] & tcp-rst != 0'	tcp.flags.reset == 1



Display filters you will use in Lab 3

- `ip.addr == 192.168.10.44` — one host, both directions
- `tcp.port == 445` — one service
- `tcp.flags.syn == 1 && tcp.flags.ack == 0` — connection attempts only
- `tcp.flags.reset == 1` — closed ports, aborted sessions
- `dns · http.request · tls.handshake.extensions_server_name contains "update"`
- `frame.len > 1000` — large frames
- `ip.addr == 192.168.10.44 && !(tcp.port == 443)` — combine conditions

The commands of this lesson

- `tcpdump -i eth1 -n -w ws44.pcap host 192.168.10.44` — capture on the mirror interface (-n: no name resolution)
 - add `-c 1000` (stop after 1000 frames) or `-G 300 -W 1` (stop after five minutes)
- `tcpdump -n -r ws44.pcap 'tcp port 445'` — read back with a filter
- `tshark -r ws44.pcap -q -z conv,tcp` — conversations in text
- `tshark -r ws44.pcap -T fields -E header=y -E separator=, -e frame.time_epoch -e ip.src -e ip.dst ... > packets.csv`
- Windows: `pktmon filter add -i 192.168.10.44 · pktmon start --capture --pkt-size 0 -f ws44.etl · pktmon stop · pktmon etl2pcap`
- Companion scripts: `capture_and_export.sh`, `capture_pktmon.ps1`

Reading a capture for monitoring — the baseline's questions

- Who are the top talkers — the servers you expect?
- Which protocols are present — any forbidden by policy (Telnet, SMBv1, clear-text FTP)?
- Connection attempts never answered (SYN without SYN-ACK)?
- One source → many ports of one host (scan)? one port → many hosts (sweep)?
- Regular-interval connections to one outside address (beaconing)?
- More data out than in (exfiltration)? DNS names nobody typed?
- Each "yes" is an anomaly candidate for Topic 5



Legal and privacy limits of capture

- A capture records other people's communications
- Shared network you do not own (residence, campus, café) without authorization = interception, Criminal Code s. 184
- Employer's network without a written mandate = unauthorized use, s. 342.1
- Legal Notice §7.2: only on systems you own or Designated Targets; never promiscuous mode on a shared network
- Every capture is personal information: keep it in the lab, delete after marking, never post a real one
- Tonight: synthetic packet list, fictional company, documentation address ranges (RFC 5737)



Videos for 4.2 — verified, each under 15 minutes

- Chris Greer — Intro to Wireshark, Lesson 4: Where do we capture network traffic? How? — ≈ 7:30 — youtube.com/watch?v=Atde35_9AAc
- Chris Greer — How to Filter Traffic, Intro to Wireshark Lesson 5 — ≈ 12:50 — youtube.com/watch?v=-HDpYR_QSFw
- Vinsloev Academy — Learn Wireshark in 10 minutes — ≈ 10:35 — youtube.com/watch?v=lb1Dw0elw0Q
- Steven Gordon — Packet Capture: tcpdump (1 of 3) — ≈ 7:45 — youtube.com/watch?v=AfKSq2jZoDw
- Watch at home; check every link with VirusTotal before clicking



Example 4.2 — Evening traffic from workstation WS-10-44

- Lab 2 tool: unusual traffic from WS-10-44 (192.168.10.44) Thursday 19:10–19:15, office empty
- Capture: monitoring server, mirror port — `tcpdump -i eth1 -n -G 300 -W 1 -w ws44.pcap host 192.168.10.44`
- Export: `tshark -T fields` → `packets.csv` (68 frames, 12 fields)
- Task: conversation summary — who, with whom, protocol, port, packets out/in, bytes, answered or not
- Rules: 10 distinct ports of one host within 60 s → PORT-SCAN-PATTERN; SYN sent, nothing received → NO-REPLY
- Output: `conversations_summary.csv` — the network-side evidence of Lab 3

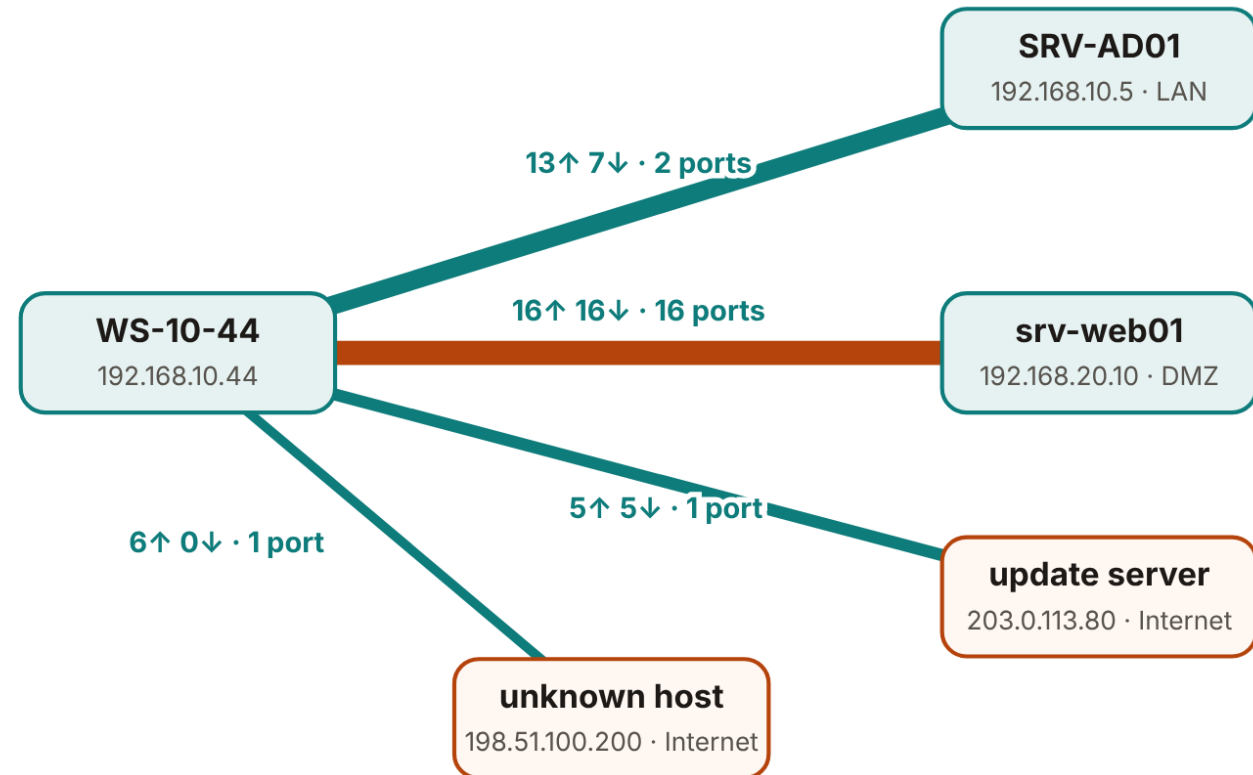
Example 4.2 — what traffic_summary.py does, step by step

- 1 read_packets(): CSV → time, addresses, protocol name, port (TCP or UDP column), flags (1/0 or True/False), length
- 2 build_conversations(): forward key (src, dst, proto, dport) / reverse key (dst, src, proto, sport)
 - reply → packets_in, synack_in, rst_in · outgoing → packets_out, syn_out
- 3 detect(): syn_out > 0 and packets_in == 0 → NO-REPLY (internal / external)
 - per (src, dst): distinct ports within 60 s ≥ 10 → PORT-SCAN-PATTERN
- 4 main(): write conversations_summary.csv, print the table and the alerts
- Python 3, standard library only; a comment before every line in the module

Example 4.2 — the result, and what it does not say

- WS-10-44 ↔ SRV-AD01:445 — 14 packets, 1,514-byte frames: the nightly backup of design files (normal)
- WS-10-44 ↔ SRV-AD01:53 — 3 DNS query/response pairs (normal)
- WS-10-44 ↔ 203.0.113.80:443 — HTTPS, 10 packets: a vendor update server (normal)
- WS-10-44 → srv-web01: SYN to 16 ports in 19 s, 13 RST, 3 SYN-ACK (22, 80, 443 open) → PORT-SCAN-PATTERN
- WS-10-44 → 198.51.100.200:4444 — 6 SYN, no reply, external → NO-REPLY
- Again: WHERE, not WHY — which process? was the designer there? did the firewall block? what is 198.51.100.200? → Lab 3

Conversations of WS-10-44 during the 5-minute capture (mirror port)



Check-in 4.2 — answer aloud

- Capture filter vs display filter?
- Why does promiscuous mode not show other hosts on a switch?
- The five values of the 5-tuple?
- The three handshake flags, and what a RST answer means?
- Which built-in Windows tool captures packets?
- In the module: 5 flash cards, 4 multiple-choice and 2 multiple-select questions with hints



Lab 3 (5 %) — Integrative Project, Part 3 — Friday, October 16 (summary)

- "Detect, diagnose and report an anomaly; propose optimizations" — published separately
- You receive: a weekend data set of the Part 1 network — Windows Security export, Linux auth.log, a 5-minute pcap and its packets.csv, the Lab 2 baseline
- 1 Detect: run `logon_failures.py` and `traffic_summary.py`
- 2 Diagnose: 4.3–4.5 and Topic 5 methods — what, when, where, who, how; compare with the baseline; raw files and pcap as evidence
- 3 Report: an incident note for the people concerned (5.4–5.5)
- 4 Optimize: update the baseline, tune the two thresholds, improve coverage, fewer false alarms (Topic 6)
- Deliverables: the two CSV summaries, a two-page report, the updated monitoring plan

Next steps and downloads

- Quiz 3 — Saturday, October 10 (5 %) — covers all material to date, published separately
- Next class (Saturday): Topic 5.1 what an anomaly is, 5.2 reviewing and sorting alerts (triage)
- Downloads (module and class14_files.zip): logon_failures.py, traffic_summary.py, export_security_4625.ps1, capture_and_export.sh, capture_pktmon.ps1
 - sample data: security_export.csv, auth.log, packets.csv; results: logon_failures_summary.csv, conversations_summary.csv
 - four figures (SVG/PNG), eleven reference tables (CSV + XLSX)
- Two sentences to leave with: "A log or a capture only exists if someone configured it before the incident." — "A flag is a detection, not a diagnosis."



Links used in this class (official sites)

- Microsoft Learn: events 4625 / 4624, Appendix L events to monitor, advanced audit policy, Get-WinEvent, wevtutil, WEF, Sysmon, Pktmon
- IETF: RFC 5424 syslog, RFC 3164 BSD syslog, RFC 7011 IPFIX · rsyslog.com · freedesktop.org journalctl · logrotate · linux-audit
- NIST SP 800-92 Guide to Computer Security Log Management · ntp.org · MITRE ATT&CK T1110, T1046
- wireshark.org (+ wiki capture/display filters, tshark man page) · tcpdump.org (+ pcap-filter) · npcap.com · zeek.org · suricata.io · opnsense.org
- laws-lois.justice.gc.ca (Criminal Code ss. 184, 342.1) · cai.gouv.qc.ca (Law 25) · cyber.gc.ca · virustotal.com
- All links, with a short explanation and a VirusTotal check button, are in the "Links" section of the module

Legal notice

- Educational material — Cyber.SoHo / Asen Grozdanov — © 2026 Cyber.SoHo — all rights reserved
- Authorized lab environment and Designated Targets only; unauthorized use may be a criminal offence (ss. 342.1, 342.2, 430(1.1), 184)
- Governed by the Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy v2.0 — sections 3, 4, 6, 6.7, 7.2, 8, 9, 10–14
- Atelier Nordik inc. and all data are fictional; external addresses from RFC 5737 documentation ranges
- Third-party names are the property of their owners; no affiliation or endorsement implied (section 5)
- <https://cyber-soho.github.io/prevmon/00.01.%20cyber.soho%20-%20legal.html>
- Cyber.SoHo Educational Hub — From the Industry to the Classroom — Building the IT's Future, Today and Together!

